

Prayer Keep AWS Deployment Guide

Step-by-step instructions for deploying the Prayer Keep frontend to S3 + CloudFront and the backend API to ECS Fargate + ALB.

Deployment model: frontend static files in S3 behind CloudFront, backend container in ECS Fargate behind an Application Load Balancer, and MongoDB hosted in Atlas.

1. Architecture

1. `app.your-domain.com` serves the frontend through CloudFront.
2. `api.your-domain.com` routes to an ALB, then to ECS Fargate.
3. MongoDB Atlas stores all application data.
4. AWS Secrets Manager stores the backend secrets.

2. Local Prerequisites

1. Install Node.js.
2. Install Docker Desktop.
3. Install AWS CLI.
4. Configure AWS credentials locally with `aws configure`.
5. Have a MongoDB Atlas connection string ready.

3. One-Time AWS Setup

3.1 Create Secrets in Secrets Manager

1. Create a secret for `MONGODB_URI`.
2. Create a secret for `JWT_SECRET`.
3. Keep the ARN for each secret. The deploy config needs both values.

3.2 Create an ECR Repository

```
aws ecr create-repository --repository-name prayer-keep-api
```

3.3 Create an ECS Cluster

```
aws ecs create-cluster --cluster-name prayer-keep-prod
```

3.4 Create the CloudWatch Log Group

```
aws logs create-log-group --log-group-name /ecs/prayer-keep-api
```

3.5 Create IAM Roles

1. Create or reuse an ECS task execution role.
2. Create or reuse an ECS task role.
3. Record both ARNs for the deploy env file.

3.6 Create the ALB and Target Group

1. Create an Application Load Balancer.
2. Create a target group with target type `ip`.
3. Set the health check path to `/api/health`.
4. Use container port `5000`.

3.7 Create the ECS Service

1. Create a Fargate service in your ECS cluster.
2. Attach it to the ALB target group.
3. Configure the correct subnets and security groups.
4. Use the service name you plan to place in the env file.

3.8 Create the Frontend S3 Bucket

```
aws s3 mb s3://prayer-keep-web-prod
```

3.9 Create the CloudFront Distribution

1. Point CloudFront to the S3 bucket origin.
2. Set `index.html` as the default root object.
3. Attach an ACM certificate if using a custom domain.
4. Record the CloudFront distribution ID.

4. Configure This Repo

1. Copy `deploy/aws/aws.env.example` to `deploy/aws/aws.env`.
2. Fill in all real AWS values.

Required Values in `deploy/aws/aws.env`

- `AWS_REGION`

- `AWS_ACCOUNT_ID`
- `ECR_REPOSITORY`
- `ECS_EXECUTION_ROLE_ARN`
- `ECS_TASK_ROLE_ARN`
- `API_BASE_URL`
- `CLIENT_ORIGIN`
- `MONGODB_URI_SECRET_ARN`
- `JWT_SECRET_ARN`
- `FRONTEND_S3_BUCKET`
- `CLOUDFRONT_DISTRIBUTION_ID`
- `ECS_CLUSTER`
- `ECS_SERVICE`

Important: `API_BASE_URL` should be your public backend URL, typically `https://api.your-domain.com/api` . `CLIENT_ORIGIN` should be your frontend URL, typically `https://app.your-domain.com` .

5. Dry Run the Deployment

Run this first to print the command sequence without changing AWS:

```
npm.cmd run aws:deploy:dry-run
```

6. Run the Real Deployment

Once the dry run looks correct, run:

```
npm.cmd run aws:deploy
```

This PowerShell script does the following:

1. Reads `deploy/aws/aws.env` .
2. Generates deploy artifacts in `deploy/aws/out` .
3. Logs Docker into Amazon ECR.
4. Builds the backend Docker image.
5. Tags and pushes the image to ECR.
6. Registers the ECS task definition.
7. Triggers an ECS service deployment.

8. Uploads the frontend bundle to S3.
9. Invalidates CloudFront.

7. Verify the Deployment

7.1 Verify the Backend

```
curl https://api.your-domain.com/api/health
```

7.2 Verify the Frontend

1. Open the frontend URL in a browser.
2. Register a test user.
3. Log in.
4. Create a prayer list.
5. Add a prayer request.

8. Troubleshooting

- If the frontend loads but API requests fail, check `API_BASE_URL`.
- If CORS errors appear, check `CLIENT_ORIGIN`.
- If ECS tasks fail to start, inspect CloudWatch logs.
- If the ALB shows unhealthy targets, verify `/api/health` and port `5000`.
- If login fails immediately after deploy, verify your secret ARNs and MongoDB URI.

9. Future Deployments

After the AWS infrastructure exists, future releases are much simpler: update the code, keep

`deploy/aws/aws.env` current, and run `npm.cmd run aws:deploy` again.

Source files for this deployment workflow live in `deploy/aws/prepare.js`, `deploy/aws/deploy.ps1`, and `deploy/aws/aws.env.example`.